Team Name : Solo_Leveler

Team Leader Name : Tanishq Jain

Problem Statement : Intelligent Candidate Discovery & Ranking — rank 100,000 candidates against a Senior AI Engineer JD the way a great recruiter would, going beyond keyword matching to understand career narratives, production evidence, and behavioral availability.

Solution Overview

- Proposed Solution: A multi-signal AI ranking system that scores 100,000 candidates against the Senior AI Engineer JD across 8 weighted dimensions — core skill depth (28%), production evidence (22%), career trajectory (15%), title fit (10%), behavioral availability (7%), experience range (7%), location (6%), and domain fit (5%). Runs on CPU, no external APIs, completes in under 60 seconds. Each top-100 candidate receives specific, fact-grounded reasoning.
- Differentiators: (1) Hard disqualifiers as score multipliers — consulting-only careers, non-technical titles, and wrong domains (CV/Speech without NLP) are penalised with 0.2–0.4x multipliers, not just soft weights. (2) Production evidence is read from career description narratives, not skill list tokens — an ML engineer at Zomato who built a ranking system scores high even without writing "vector database" in their summary. (3) Behavioral availability (last active, response rate, notice period, GitHub) is modelled explicitly — a perfect-on-paper candidate who is unreachable is deprioritised.

JD Understanding & Candidate Evaluation

- Key JD Requirements Extracted: (1) Must-haves — production experience with embeddings-based retrieval (sentence-transformers, BGE, E5, OpenAI), vector databases (Pinecone, Weaviate, Qdrant, FAISS, OpenSearch), hybrid search, strong Python, and rigorous evaluation frameworks (NDCG, MRR, MAP, A/B testing). (2) Experience range: 5-9 years in applied ML at product companies. (3) Location: Pune, Noida, or major Indian cities; open to relocation. (4) Explicit disqualifiers: consulting-only careers (TCS, Infosys, Wipro etc.), non-technical titles, CV/Speech/Robotics without NLP, pure research without production deployment, recent LangChain-only experience. (5) Soft signals: short notice period (sub-30 days preferred), active on platform, GitHub activity.
- Candidate Signals Beyond Keywords: We evaluate (a) Career narrative — job description text is scanned for 20+ production signals (“deployed”, “real users”, “A/B test”, “latency”, “index refresh”) rather than just skill list tokens. (b) Trajectory quality — months spent at product companies vs consulting firms, and whether ML roles were in production contexts. (c) Behavioral availability — last_active_date, recruiter_response_rate, notice_period_days, interview_completion_rate, and github_activity_score together model whether a candidate is actually reachable and hirable today. (d) Domain alignment — NLP/IR background vs CV/Speech-only, checked against both skills and career descriptions.

Ranking Methodology

- Retrieval, Scoring, Ranking: All 100,000 candidates are loaded from JSONL and scored in a single pass. Each candidate is scored independently in O(1) time via substring matching against a curated ontology of ~70 core skill terms and 20 production-evidence signals. Candidates are sorted by final score descending; the top 100 are written to CSV with per-candidate reasoning. No pre-filtering, no vector index, no approximate nearest neighbour — every candidate is fully evaluated.
- Models, Algorithms, Heuristics: Pure Python — no ML models, no embeddings, no external libraries beyond stdlib. Eight hand-crafted scoring functions derived from deep JD reading: score_core_skills (proficiency-weighted ontology matching), score_production_evidence (narrative signal detection), score_career_trajectory (product-vs-consulting month ratio), score_title_fit (regex on current title), score_experience_range (piecewise function on YoE), score_location (city lookup), score_behavioral_signals (composite of 7 redrob_signals fields), score_domain_fit (NLP/IR vs CV/Speech domain check). Honeypot detection uses timeline consistency checks and expert-skill inflation heuristics.
- Combining Signals: final_score = weighted_sum(8 components) x disqualifier_multiplier. Weights: core_skills 28%, production_evidence 22%, career_trajectory 15%, title_fit 10%, behavioral 7%, experience_range 7%, location 6%, domain_fit 5%. Disqualifier multipliers are applied multiplicatively on top: consulting-only career x0.30, non-technical current title x0.20, wrong domain x0.40. This ensures disqualifiers actually fire rather than being washed out by strong skill matches — the key design insight from reading the JD carefully.

Explainability & Data Validation

- Ranking Explainability: Each of the top 100 candidates receives a 1-2 sentence reasoning generated by build_reasoning() in reasoning.py. Reasoning is rank-consistent in tone (glowing for top 10, measured for mid-tier, honest about weaknesses at the tail). It references specific facts: current title, company, years of experience, named skills from the actual profile, behavioral signals (notice period, GitHub score, last active). Tone is adjusted by rank tier so a rank-5 candidate does not receive the same language as a rank-90 candidate.
- Preventing Hallucinations: Reasoning is constructed programmatically from actual candidate fields — it never invents facts. Named skills in reasoning are drawn only from the candidate's skills list (top 3 matching AI-relevant skills). Company and title come directly from profile fields. Concern flags (notice period, last active days, response rate) are computed from redrob_signals values, not inferred. Since no LLM generates the reasoning, there is no hallucination risk at inference time.
- Handling Bad Profiles: detect_honeypot() flags candidates with impossible timelines (start year before 1990 or after 2026, job duration over 600 months), stated YoE more than 2.5x total career history months, or 15+ "expert" skills. Flagged candidates receive score = 0.0 and are never included in the top 100. Our run detected and suppressed 21 honeypots out of 100,000 candidates, with zero honeypots in the final top 100 — well within the 10% disqualification threshold.

End-to-End Workflow

- Step 1 — JD Analysis (offline, manual): Read job_description.docx thoroughly. Extract must-have skills, explicit disqualifiers, domain requirements, experience range, location preferences, and behavioral signals that matter. Encode these as Python constants in features.py (CORE_SKILLS, BAD_TITLES, CONSULTING_FIRMS, etc.). Step 2 — Candidate Loading: rank.py loads all 100,000 candidates from candidates.jsonl line-by-line into memory (~17 seconds). Step 3 — Scoring: Each candidate is passed to score_candidate() which runs 8 scoring functions + honeypot detection and returns a (score, breakdown) tuple (~30 seconds for 100K). Step 4 — Sorting: All scored candidates are sorted by score descending; top 100 are selected. Step 5 — Reasoning Generation: For each of the top 100, build_reasoning() constructs a rank-consistent, fact-grounded 1-2 sentence justification. Step 6 — CSV Output: Results are written to team_redrob_ai.csv in the required format (candidate_id, rank, score, reasoning). Total runtime: ~47 seconds on CPU.

System Architecture

- **Input Layer:** candidates.jsonl (100,000 records) + job_description.docx (read offline, encoded as constants)
- **Scoring Engine (features.py):** 8 pure-Python scoring functions run per candidate — core skills, production evidence, career trajectory, title fit, experience range, location, behavioral signals, domain fit — plus detect_honeypot(). All O(1) per candidate via substring matching against curated ontologies.
- **Ranking Layer (rank.py):** Weighted sum of component scores x disqualifier multiplier. Python sort() on all 100K scored candidates. Top 100 selected. No GPU, no network, no external dependencies.
- **Reasoning Layer (reasoning.py):** Programmatic template fills actual profile facts (title, company, YoE, named skills, signal values) into rank-consistent sentence patterns. No LLM at inference time.
- **Output Layer:** team_redrob_ai.csv — 100 rows, columns: candidate_id, rank, score, reasoning. Validated with official validate_submission.py.

Results & Performance

- Ranking Quality: Top 10 candidates are Senior AI/NLP/ML Engineers at Indian product companies — Salesforce, Zomato, Ola, Netflix, Paytm, Niramai — with hands-on experience in FAISS, Pinecone, Qdrant, Weaviate, OpenSearch, LoRA, and semantic search. No HR Managers, no Content Writers, no CV-only engineers appear in the top 100. The sample submission provided in the challenge bundle ranked an HR Manager at #1 — our system correctly scores such profiles near zero due to the title-fit disqualifier. 21 honeypots were detected and suppressed; 0 appear in top 100 (challenge threshold is 10%). Score range: 0.952 (rank 1) to 0.807 (rank 100), providing meaningful differentiation across the shortlist.
- Compute Constraints Met: Total runtime 47 seconds (limit: 300s). Memory: ~2GB peak for 100K candidates in memory (limit: 16GB). No GPU used. No network calls during ranking — zero external API calls. No dependencies beyond Python stdlib — requirements.txt is intentionally empty. Reproduce command: python rank.py --candidates ./candidates.jsonl --out ./team_redrob_ai.csv. Pre-computation: none required. The ranking step and the full pipeline are the same single command.

Technologies Used

- Python 3.11 (stdlib only) — chosen because the compute constraint (no network, no GPU, 5min, 16GB) makes heavyweight ML frameworks counterproductive. Pure Python with json, csv, gzip, re, datetime modules is sufficient and maximally portable. No pandas, no numpy, no scikit-learn, no transformers library. PptxGenJS (Node.js) — used offline to generate the presentation deck. LibreOffice + pdftoppm — for PDF conversion and visual QA of slides. Claude (Anthropic) — used as a coding assistant for architecture discussion and code review during development; no candidate data was sent to any LLM. Official validate_submission.py — run to verify submission format compliance before finalising.

Submission Assets

GitHub Repository: https://github.com/tanishq7389/redrob-ranker — contains rank.py, src/features.py, src/reasoning.py, README.md, requirements.txt, submission_metadata.yaml, and the output CSV. Single reproduce command: python rank.py --candidates ./candidates.jsonl --out ./team_redrob_ai.csv | Ranked Output CSV: team_redrob_ai.csv — top 100 candidates ranked for the Senior AI Engineer JD, with candidate_id, rank, score, and reasoning columns. Validated with official validate_submission.py (0 errors). | Sandbox / Demo: https://codesandbox.io/p/github/tanishq7389/redrob-ranker/main?workspaceId=ws_VKRvWRM8f57nhZDChVLLLs — e.g. Google Colab notebook or HuggingFace Space where the ranker can be run on a sample of candidates end-to-end. | Approach Deck: This presentation (Idea_Submission_Template_Redrob_filled.pptx) explaining the full methodology. | AI Tools Declared: Claude (Anthropic) used for coding assistance. All engineering decisions made by the team.

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU